



Real-Time Ship Detection using YOLOv8

A real-time **Ship Detection System** using a custom-trained **YOLOv8 object detection model**. The system can detect ships from a webcam, video file, or RTSP stream and display the detected objects with bounding boxes and confidence scores.



Features

- Real-time ship detection
- Webcam support
- Video file support
- RTSP stream support
- Custom-trained YOLOv8 model
- Automatic GPU/CPU detection
- Configurable confidence threshold
- Configurable IoU threshold
- Real-time FPS display
- Option to save annotated detection videos

The detection script uses YOLO inference with configurable image size, confidence, IoU and device parameters.



Technologies Used

- Python
- YOLOv8
- Ultralytics
- OpenCV
- PyTorch
- Torchvision

The project requirements include Ultralytics, OpenCV, PyTorch and Torchvision.



Project Structure

```
Ship-Detection/  
|  
├─ best.pt  
├─ webcam_detect.py  
├─ requirements.txt  
└─ README.md
```

Files

File	Description
<code>best.pt</code>	Trained YOLOv8 model weights
<code>webcam_detect.py</code>	Real-time ship detection script
<code>requirements.txt</code>	Required Python packages
<code>README.md</code>	Project documentation

Installation

1. Clone the repository

```
git clone https://github.com/your-username/ship-detection.git
cd ship-detection
```

2. Create a virtual environment

```
python -m venv venv
```

Activate it on Windows:

```
venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

The provided requirements specify:

```
ultralytics>=8.2.0
opencv-python>=4.9.0
torch>=2.2.0
torchvision>=0.17.0
```

For GPU acceleration, install the CUDA-enabled PyTorch build appropriate for your system.

Usage

Webcam Detection

Place `best.pt` in the project directory and run:

```
python webcam_detect.py --weights best.pt
```

By default, the program uses camera `0`.

```
python webcam_detect.py --weights best.pt --camera 0
```

The program displays the detection window and shows the current FPS. Press `q` to stop detection.

Detect Ships in a Video

You can also use a video file instead of a webcam:

```
python webcam_detect.py --weights best.pt --source video.mp4
```

The script also supports RTSP streams through the `--source` option.

Save Detection Output

To save the annotated video:

```
python webcam_detect.py --weights best.pt --source video.mp4 --save  
output.mp4
```

The program automatically creates a video writer when the `--save` option is provided.

Detection Parameters

The detection settings can be customized using command-line arguments.

```
python webcam_detect.py  
  --weights best.pt  
  --camera 0  
  --imgsz 640
```

```
--conf 0.5
--iou 0.45
```

Parameter	Description	Default
<code>--weights</code>	Path to trained YOLO model	<code>runs/train/ship_yolov8s/weights/best.pt</code>
<code>--camera</code>	Camera index	<code>0</code>
<code>--source</code>	Video/RTSP source	None
<code>--imgsz</code>	Input image size	<code>640</code>
<code>--conf</code>	Detection confidence threshold	<code>0.5</code>
<code>--iou</code>	IoU threshold for NMS	<code>0.45</code>
<code>--device</code>	CPU/GPU device	Auto
<code>--save</code>	Output video path	None

These parameters are defined directly in the detection script.

How It Works

```
Camera / Video / RTSP
    ↓
Video Frame
    ↓
YOLOv8
    ↓
Ship Detection
    ↓
Bounding Boxes + Confidence
    ↓
Display Result
    ↓
Optional Video Saving
```

For every frame, the system runs YOLO inference and plots the detection results onto the frame before displaying them.

GPU Support

The program automatically checks whether CUDA is available.

If a CUDA-compatible GPU is available:

GPU → YOLO inference

Otherwise:

CPU → YOLO inference

You can also manually select the device:

```
python webcam_detect.py --weights best.pt --device cpu
```

or:

```
python webcam_detect.py --weights best.pt --device 0
```

The device is automatically selected in the script when `--device` is not specified.

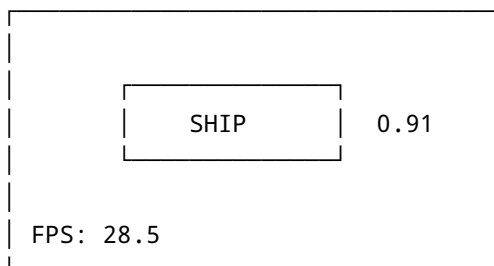


Output

The system displays:

- Detected ship bounding boxes
- Detection confidence
- Real-time FPS
- Annotated video feed

Example:



Future Improvements

- Add multiple ship categories
- Improve detection accuracy with a larger dataset
- Add object tracking
- Count detected ships

- Add distance estimation
 - Add automatic ship classification
 - Create a web-based monitoring dashboard
 - Add alert notifications for detected ships
 - Deploy the model on edge devices
 - Improve performance for low-light and adverse weather conditions
-

Author

Nihal H B

Computer Science & Data Science
CMR Institute of Technology

License

This project is intended for educational and research purposes.